COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Oct 14, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Bal, Kevin Ranz J. Braganza, Ralph Angelov F.	**INSTRUCTOR:**	Engr. Jimlord M. QuejadoILO A & ILO B:

Hands-on Activity 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Screenshot of the code

finals > Hands-on Activities > HoA 13.1 > ILO A > main.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <string>
void print(int n, const std::string &str)
{
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main()
{
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Output of the code

Bal's Output

```
Printing integer: 10
Printing string: T.I.P.
```

Braganza's Output

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

Bal's Analysis

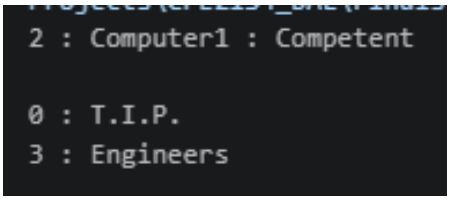
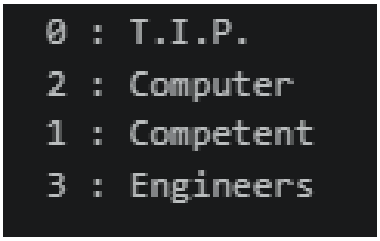
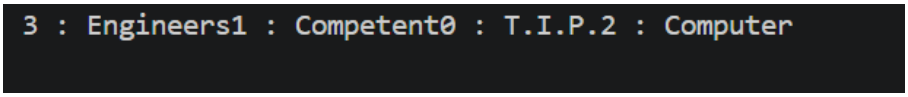
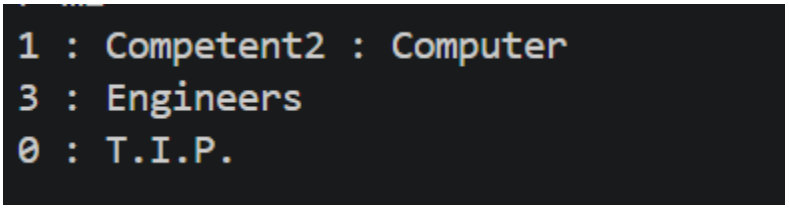
When this program runs, the main function creates and initializes a thread named "t1" and passes the arguments that has been declared which are the name of the function, integer value and the string text that has been moved to the function where

	the output displays the given integer 10 and the string named “T.I.P.”. The join function waits for the t1 function name that comes from the thread STL to finish its operation to prevent the early closure of the program. **Braganza’s Analysis** This code illustrates the basic principle of multithreading in C++. This means that in order to start the execution of a particular function, we will create an additional thread. Function print() takes two arguments n and str. The parameters contain the values of 10 and "T.I.P.", respectively. In main(), std::thread t1(print, 10, "T.I.P."); generates a new thread t1 which independently executes the print() function. Statement t1.join() makes sure that the main thread is waiting until the completion of execution of t1. Without the usage of t1.join(), the program may try to exit even though the created thread is not yet done with its execution.

Part 2: Multi-Threaded Example	
Screenshot of the code	

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str)
{
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main()
{
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"};
    std::vector<std::thread> threads;
    for (int i = 0; i < s.size(); i++)
    {
        threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads)
    {
        th.join();
    }
    return 0;
}
```

Output of the code	**Bal's Output** **1st Run**  **2nd Run**  **Braganza's Output** **1st Run**  **2nd Run** 
Analysis	**Bal's Analysis** In this multi-threaded part, there are two for loops in the program which performs the initialization of the thread for the four concurrent threads based on the index of the strings of the array. The second for loop performs iteration to the threads on the vector and apply join function for each of the thread, which waits for each thread to complete their assigned task, from the first procedure, the output display and show different arrangements of the threads that display their assigned because of the multiple threads that perform one operation concurrently. **Braganza's Analysis** This program shows the concept of multithreading in C++ through the creation of four threads executing the print() function with various inputs of integers and strings. In the first run of the program, the output comes out in the following manner: **3 : Engineers1 : Competent0 : T.I.P.2 : Computer**

	It is evident from this that there has been a difference in the order of execution and the order of displaying the output of each thread. In the second run, the outputs appear in the following form: **1 : Competent2 : Computer** **3 : Engineers** **0 : T.I.P.** The differences in the order of the messages come about due to the fact that each thread is scheduled independently by the OS; hence, there is no guarantee that their order of execution and printing will be as per the order in which they were created.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.
Questions:

Bal's Answers

1. Write a definition of multithreading and its advantages/disadvantage
Multithreading means breaking a large task into smaller threads that can run at the same time while sharing the same memory. This makes a program faster and more responsive because the threads share the computer's CPU power and memory inside a single program. By splitting up the work, the entire process gets done much faster than if only one thread did everything in a line. However, this approach can cause bugs because the threads share the exact same space. For example, an issue happens when one thread deletes a file or a piece of data while another thread is still in the middle of reading it, causing the whole program to freeze or crash. It is also harder to write the code because you have to carefully manage how each thread uses the shared parts without crashing.

2. Rationalize the use of multithreading by providing at least 3 use-cases.
It is best to use multithreading when a program has to wait around for a task to finish, allowing other tasks to get done instead of sitting idly. Examples of this include printing multiple documents because one thread can send data to the printer while another thread lets you keep typing in your text editor. It is also very helpful for downloading updates in a video game, where a program can fetch new game files in the background while letting you browse the main menu at the same time. Lastly, it works great for loading images in a social media app because while one thread waits for pictures to load from the internet, another thread can keep the app scrolling without freezing.

3. Differentiate between parallelism and concurrency.
Concurrency means letting multiple tasks take turns on a single computer path while sharing the same memory. On the other hand, parallelism happens when a computer runs separate tasks at the exact same moment across different CPU cores without needing them to talk to each other. To make it simple, concurrency is like having two lines of people waiting to share one coffee machine. Parallelism is like having two lines of people using two separate coffee machines at the exact same time.

Braganza's Answer

1. Write a definition of multithreading and its advantages/disadvantage
Multithreading is defined as the utilization of several threads within a C++ program in which each thread manages its own task. Through multithreading, the efficiency of the program is increased since various tasks can be performed concurrently and in parallel by using several cores of the processor if such hardware support exists. The benefits of multithreading involve improved CPU usage, greater responsiveness, and faster execution of relevant tasks. The drawbacks associated with multithreading are increased programming complexity, greater memory consumption, and potential problems due to data race conditions among other things.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Editing and Rendering of Videos: Multithreading allows the software to perform different operations on the video in parallel. This might help in reducing the processing time required to render the video.
Web Server: Multithreading allows the server to service different client requests in parallel such that there is no need for each client to wait for the other client's request to get serviced.
Games and Simulations: Multithreading allows execution of different tasks like physical calculations, audio operations, and background calculations on separate threads helping improve performance and responsiveness.

3. Differentiate between parallelism and concurrency.
Concurrency is the ability of a program to handle more than one task within overlapping time frames. These tasks could be alternated by the program or done concurrently by multiple CPU cores. Parallelism is the ability of running more than one task at once, usually achieved by using more than one CPU cores. A case in which a program performs tasks like taking user inputs while downloading files is an example of concurrency. On the other hand, when programs process data from multiple chunks of a dataset simultaneously, it is an example of parallelism. Both techniques can improve program efficiency, but their effectiveness depends on the tasks being performed and the available hardware.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that

    - o Display
    - o T1.join()
    - o Display
    - o T2.join()
    - o Display
    - o T3.join(

- Provide an analysis based on the outputs of the multi-threading exercise.

Screenshot of the code	**Bal's Code** Finals > Hands-on Activities > HoA 13.1 > ILO B > main.cpp > main() **Braganza's Code** CPE010_HOA13p1_Braganza_sc3.cpp > ...

```cpp
#include <iostream>
#include <thread>

//global variable integer
int sum = 0;

// function that will take an integer parameter and add that value to the global variable.
void add(int value)
{
    sum += value;
}

int main()
{
//creating the three threads with name and parameters inside to pass the void function to the threads
    std::thread t1(add, 65);
    std::thread t2(add, 40);
    std::thread t3(add, 63);

    std::cout << "Displaying the sum of three combined threads" << std::endl;
    std::cout << "Thread 1: " << sum << std::endl;
    t1.join();
    std::cout << "Thread 2: " << sum << std::endl;
    t2.join();
    std::cout << "Thread 1: " << sum << std::endl;
    t3.join();
    std::cout << "Total Threads: " << sum << std::endl;
    return 0;
}
```

```cpp
#include <iostream>
#include <vector>
#include <thread>

int globalSum = 69;

void add(int add){
    globalSum += add;
    std::cout<<globalSum<<std::endl;
}

int main(){

    std::thread T1(add, 21);
    std::thread T2(add, 14);
    std::thread T3(add, 67);

    std::cout<<globalSum<<std::endl;
    T1.join();
    std::cout<<globalSum<<std::endl;
    T2.join();
    std::cout<<globalSum<<std::endl;
    T3.join();
    std::cout<<globalSum<<std::endl;

return 0;

}
```

Output of the code	**Bal's Output**
	1st Run
	`Displaying the sum of three combined threads`
	`Thread 1: 0`
	`Thread 2: 168`
	`Thread 1: 168`
	`Total Threads: 168`
	2nd Run
	`Displaying the sum of three combined threads`
	`Thread 1: 0`
	`Thread 2: 168`
	`Thread 3: 168`
	`Total Threads: 168`
	
	Braganza's Output
	1st Run
	`69`
	`83171`
	`104`
	
	`171`
	`171`
	`171`
	
	2nd Run
	`69`
	`90`
	`104`
	`171`
	`171`
	`171`
	`171`

Analysis	**Bal's Analysis** The program creates three threads that add different values to the global integer variable named "sum", and creates a void add function for the operation with the sum and parameters inside it. The join() function makes the main thread wait for a specific thread to finish, but it does not control the order in which the threads will be executed. The add(0 function updates the total whenever a thread runs. After all the threads have been executed, it resulted in a total of 168. **Braganza' Analysis** In my code implementation, both outputs show that multithreaded programs can give different outputs according to how the operating system schedules the threads of the programs. The function join() waits for a particular thread to be completed by the main thread. However, the function join() does not dictate the order of execution of threads. Due to the presence of data races because all three threads manipulate the same global variable without synchronization, there is no guarantee that the program will always give the desired output of 171.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Screenshot of the code	Modified code from Hands-on Activity 8.1 Sorting Algorithms Pt2

main.cpp

CPE010_HOA13p1_Braganza_sc4.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono> // Modified Added for measuring execution time
#include "CPE010_HOA13p1_Braganza_sc4.h" //Modified the name

const int SIZE = 100;

int main(){

    // Array of size 100 to hold random integers
    int arr[SIZE];

    // MODIFIED
    //Added a second array to keep the original unsorted values
    // for the multithreaded version
    int arr2[SIZE];

    srand(time(0));

    for (int i = 0; i < SIZE; i++){
        arr[i] = rand()%100;

        // MODIFIED
        //Copy each random number into the second array
        arr2[i] = arr[i];
    }

    std::cout << "Original Array:" << std::endl;
    displayArray(arr, SIZE);

    // Original single-threaded Merge Sort
    // MODIFIED
    //Measure the execution time using chrono
    auto start1 = std::chrono::high_resolution_clock::now();
    mergeSort(arr, 0, SIZE - 1);
    auto end1 = std::chrono::high_resolution_clock::now();
    auto duration1 = std::chrono::duration_cast<std::chrono::microseconds>(end1 - start1);

    std::cout << "\n\nSingle-Threaded Merge Sort:" << std::endl;
    displayArray(arr, SIZE);
    std::cout << "\nExecution Time: " << duration1.count() << " microseconds" << std::endl;

    auto start2 = std::chrono::high_resolution_clock::now();
    parallelMergeSort(arr2, 0, SIZE - 1);
    auto end2 = std::chrono::high_resolution_clock::now();
    auto duration2 = std::chrono::duration_cast<std::chrono::microseconds>(end2 - start2);

    std::cout << "\n\nMultithreaded Merge Sort:" << std::endl;
    displayArray(arr2, SIZE);
    std::cout << "\nExecution Time: " << duration2.count() << " microseconds" << std::endl;

    return 0;
}
```

Mergesort.h

CPE010_HOA13p1_Braganza_sc4.h > ...

```cpp
#ifndef CPE010_HOA13P1_BRAGANZA_SC4_H
#define CPE010_HOA13P1_BRAGANZA_SC4_H

#include <iostream>
#include <thread> // Added to use std::thread for multithreading

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
}

/*
A.2. Merge Sort
General Algorithm
The general pseudo-code for the merge sort technique is given below.
Declare an array Arr of length N
If N=1, Arr is already sorted
If N>1,
Left = 0, right = N-1
Find middle = (left + right)/2
Call merge_sort(Arr,left,middle) =>sort first half recursively
Call merge_sort(Arr,middle+1,right) => sort second half recursively
Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
Exit
*/

// Merge Sort (CREDITS TO PROGRAMIZ)
```

```
// Merge two subarrays L and M into arr
void merge(int arr[], int p, int q, int r)
{
    // Create L <- A[p..q] and M <- A[q+1..r]
    int n1 = q - p + 1;
    int n2 = r - q;

    int* L = new int[n1];
    int* M = new int[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[p + i];

    for (int j = 0; j < n2; j++)
        M[j] = arr[q + 1 + j];

    // Maintain current index of sub-arrays and main array
    int i = 0, j = 0, k = p;

    // Until we reach either end of either L and M, pick the smaller element
    // and place it in the correct position in the main array
    while (i < n1 && j < n2){
        if (L[i] <= M[j]){
            arr[k] = L[i];
            i++;
        }
        else{
            arr[k] = M[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements from L
    while (i < n1){
        arr[k] = L[i];
        i++;
        k++;
    }
```

```
    // Copy the remaining elements from M
    while (j < n2){
        arr[k] = M[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] M;
}
```

The only modified part of the mergesort.h file

```
// MODIFIED PART
// Original single-threaded Merge Sort
void mergeSort(int arr[], int l, int r)
{
    if (l < r){
        // Find the middle point to divide the array
        int m = l + (r - l) / 2;

        // Sort both halves recursively using the original algorithm
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

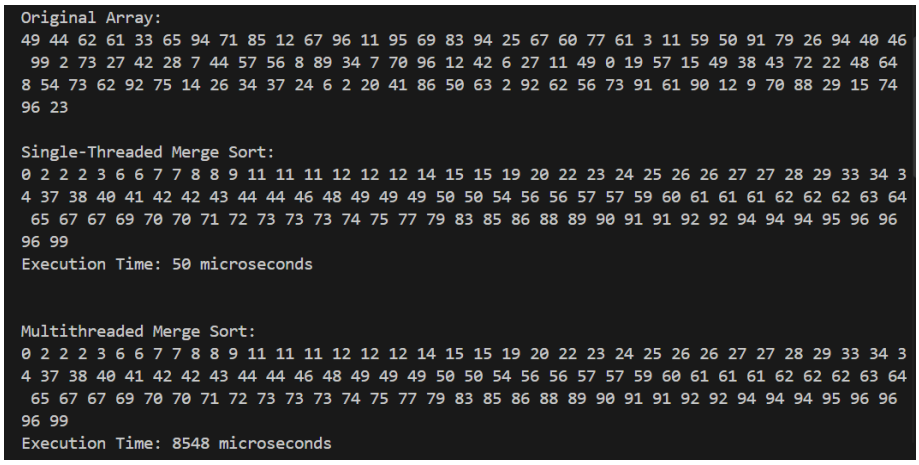
        // Merge the sorted halves
        merge(arr, l, m, r);
    }
}

// MODFIED PART
void parallelMergeSort(int arr[], int l, int r)
{
    if (l < r){
        int m = l + (r - l) / 2;
        // MODFIED
        // Create two threads to sort separate halves of the array
        // Each thread runs the original mergeSort() function
        std::thread T1(mergeSort, arr, l, m);
        std::thread T2(mergeSort, arr, m + 1, r);

        // MODFIED
        // Wait for both threads to finish sorting their halves
        T1.join();
        T2.join();

        // Merge the sorted subarrays
        merge(arr, l, m, r);
    }
}

#endif
```

Output of the code	 **Braganza's Output**
Analysis	**Bal's Analysis** This program compares how fast a computer can sort a list of 100 random numbers using two different methods, a standard single-threaded way and a new multithreaded method. The standard method handles the whole list on a single path, meaning it has to finish sorting the left side before it can even start on the right side. The multithreaded method splits the work by creating two separate paths at the same time, where one path handles the left half of the numbers while the other path handles the right half simultaneously. The program tracks the exact execution time in microseconds for both methods, revealing that for a small list of 100 numbers, the single-threaded version is often faster because creating and managing extra paths takes up extra computer effort that outweighs the time saved by sorting at the same time. **Braganza's Analysis** The results show that both the single-threaded and multithreaded versions successfully sorted the array in ascending order. However, the single-threaded version took only 50 microseconds, while the multithreaded version took 8,548 microseconds. This shows that multithreading does not always make an algorithm faster because creating and managing threads also requires time. Since the program only sorts 100 integers, the overhead of multithreading was greater than the time saved by processing parts of the array concurrently. With a larger array, multithreading may provide better performance, although the actual result depends on the implementation and the computer's hardware.

Conclusion
Bal's Conclusion In this activity, It teaches and introduce the concepts, purpose and importance of the multithreading, it provides

me a good knowledge and new learning about the new STL named “thread” and how the join and the implementation works on each procedure that provides me and Ralph knowledge about it even we’re unsure how to start from scratch. This STL provides the concurrency, but it was depending on the hardware, like it can execute different kinds of threads using the parallelism/multi-threads in once but don’t know who will be the one will be executed because the each threads that had been created are racing each other who will be the one will be displayed in the console while the single thread executes line-by-line sequentially. If line 2 takes 5 seconds, line 3 must wait just to be executed. The supplementary activity demonstrates the implementation will be using what knowledge we earned on the discussion last time, it showing how the join affects the shared global variables and how the data would be splitted through threads that can give the given dataset of the algorithm the speed of the execution of it.

Braganza’s Conclusion

In this hands-on activity I was introduced to the concept of multithreading and the way to apply std::thread, join(), and chrono in C++ for concurrent task execution and measurement of its execution time. To perform multithreading, I used the algorithm of Merge Sort by performing sorting on two halves of an array using different threads. The method helped me learn how to change the code of an already existing algorithm without changing its basic idea. In the additional activity, I compared single and multithreaded algorithms using the same set of random numbers. Both programs managed to sort the array, but the single-threaded algorithm took 50 microseconds, and the multithreaded one took 8,548 microseconds. It means that multithreading does not necessarily speed up a program execution and it isn’t always useful for every application.

Overall, I believe that I did quite well in this activity since I managed to implement multithreading into an algorithm and analyze its output. At the same time, I have realized that it is necessary to evaluate the performance of the programs rather than just presume that a multithreaded application would always work faster. In the future, however, I will have to enhance my knowledge regarding thread management and work on testing algorithms with more data to see how the performance changes in such cases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Search. Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for explaining the concepts in the most interpreted way that we could understand it including generating an idea on the Part C where giving us a simple idea that doesn’t align to the given activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.